

ISTQB · CTFL

Certified Tester Foundation Level

v 4.0

PERSONAL STUDY GUIDE

6 Chapters · Glossary · Common Mistakes

Table of Contents

1. Chapter 1 — Fundamentals of Testing

2. Chapter 2 — Testing Throughout the SDLC

3. Chapter 3 — Static Testing

4. Chapter 4 — Test Analysis and Design

5. Chapter 5 — Managing Test Activities

6. Chapter 6 — Test Tools

7. Glossary

8. Common Mistakes

Chapter 1 - Fundamentals of Testing

The big idea: Testing is much more than just clicking around and hoping something breaks. It's a structured, professional discipline that helps build quality into software — not just find bugs at the end.

Cognitive Levels in This Chapter

The ISTQB exam classifies every learning objective at one of three cognitive levels (based on Bloom's Taxonomy):

Level	Keyword	What the exam expects
K1	Remember	Recall or recognize a term, concept, or fact — no explanation needed
K2	Understand	Explain, differentiate, compare, or give examples — go beyond simple recall
K3	Apply	Use a technique on a concrete scenario to derive a result

Learning Objectives for Chapter 1

LO	K	Topic
FL-1.1.1	K1	Identify typical objectives of testing
FL-1.1.2	K2	Differentiate testing from debugging
FL-1.2.1	K2	Exemplify why testing is necessary
FL-1.2.2	K1	Recall the relationship between testing and quality assurance
FL-1.2.3	K2	Distinguish between error, defect, failure, and root cause
FL-1.3.1	K2	Explain the seven testing principles
FL-1.4.1	K2	Summarize the different test activities and tasks
FL-1.4.2	K2	Explain the impact of context on the test process
FL-1.4.3	K2	Differentiate testware that supports test activities
FL-1.4.4	K2	Explain the value of maintaining traceability between the test basis and testware
FL-1.5.1	K2	Explain the role of human psychology in testing
FL-1.5.2	K1	Identify the skills required for testing
FL-1.5.3	K2	Distinguish the mindset required for testing from the mindset required for developing

Exam tip: Chapter 1 is almost entirely K2 — focus on being able to *explain* and *distinguish*, not just name. The only K1 items are FL-1.1.1, FL-1.2.2, and FL-1.5.2. No K3 in this chapter.

What is Testing, really? — 💡 Understand (K2)

Software testing is a set of activities to **discover defects**, **evaluate quality**, and **provide information** so stakeholders can make informed decisions. Testing has seven official objectives (FL-1.1.1 — K1, know them all):

1. Evaluate work products — requirements, user stories, designs, code
2. Trigger failures and find defects
3. Ensure required coverage of the test object
4. Reduce the risk of inadequate software quality
5. Verify that specified requirements have been fulfilled
6. Verify compliance with contractual and legal requirements
7. Provide information to stakeholders to allow informed decisions

But here's what many people get wrong about testing:

- Testing is **not only** about finding bugs — it also builds confidence that the software works
- Testing is **not only** about running the software — it includes reviewing documents, designs, and requirements (static testing) too
- Testing involves **both verification** ("are we building the product right?") and **validation** ("are we building the right product?")

Testing vs. Debugging — not the same thing! — 💡 Understand (K2)

Activity	Who does it	What it does
Testing	Testers	Finds failures and defects
Debugging	Developers	Finds the cause, fixes it, removes the defect

Testing triggers the failure → debugging finds and fixes the root cause.

Why is Testing Necessary? — 💡 Understand (K2)

Software defects happen because humans make mistakes. Those mistakes create defects, and defects — when executed — cause failures. And failures can cost money, reputation, or even lives.

The error → defect → failure chain — 💡 Understand (K2)

- **Error:** A human mistake (e.g., a developer misunderstands a requirement)
- **Defect:** The flaw that the error introduces into the code or document
- **Failure:** What happens when that defect is executed and the system behaves incorrectly
- **Root cause:** The underlying reason why the error happened in the first place

Not all defects cause failures — some only fail under specific conditions, and some may never cause a failure at all.

Testing vs. Quality Assurance (QA) — also not the same! — 🧠 Remember (K1)

- **Testing** is **product-oriented** and **corrective** — it finds defects in what was built
- **QA** is **process-oriented** and **preventive** — it improves the process to prevent defects from happening

Think of QA as setting up a good kitchen process, and testing as tasting the dish to check if it came out right.

The 7 Testing Principles — 💡 Understand (K2)

These are the foundation of everything in ISTQB. Know them cold.

#	Principle	What it means in practice
1	Testing shows presence of defects, not their absence	Passing all tests doesn't mean the software is bug-free
2	Exhaustive testing is impossible	You can't test everything — use risk and priorities to focus
3	Early testing saves time and money	The later a defect is found, the more expensive it is to fix
4	Defects cluster together	A small number of modules usually contain most defects (Pareto principle)
5	Tests wear out	Running the same tests repeatedly finds fewer new defects — vary and update them
6	Testing is context dependent	Testing a banking app is very different from testing a game
7	Absence of defects is a fallacy	A bug-free system that doesn't meet user needs is still a failure

Test Activities & Tasks — 💡 Understand (K2)

A test process is made up of these activities — they often happen iteratively, not strictly in sequence:

1. **Test planning** — Define objectives and approach
2. **Test monitoring and control** — Track progress; take corrective actions if needed
3. **Test analysis** — *What* to test? Analyze the test basis to identify test conditions
4. **Test design** — *How* to test? Turn conditions into test cases and define test data
5. **Test implementation** — Prepare everything needed: test scripts, test suites, environment setup
6. **Test execution** — Run the tests, log results, report anomalies
7. **Test completion** — Wrap up: archive testware, write completion report, capture lessons learned

Testware — what each activity produces — 💡 Understand (K2)

Activity	Output examples
Planning	Test plan, risk register
Monitoring & control	Progress reports
Analysis	Test conditions, defect reports
Design	Test cases, test charters, test data requirements
Implementation	Test scripts, test suites, test execution schedule
Execution	Test logs, defect reports
Completion	Test completion report, lessons learned

Traceability — 💡 Understand (K2)

Good traceability links test cases back to requirements and risks. This helps: - Measure coverage - Analyze the impact of changes - Support audits and governance reporting

Roles in Testing — 💡 Understand (K2)

Two principal roles exist:

- **Test management role** — Focuses on planning, monitoring, control, and completion. Responsible for the test process and team leadership.
- **Testing role** — Focuses on the technical side: analysis, design, implementation, and execution.

One person can hold both roles at the same time, depending on the project context.

Essential Skills for Testers — 🧠 Remember (K1)

Good testers need a mix of: - **Testing knowledge** — knowing how to apply techniques effectively - **Analytical and critical thinking** — to find what others miss - **Good communication** — testers often deliver bad news; doing it constructively matters - **Domain knowledge** — understanding the business context - **Technical knowledge** — using tools efficiently

Whole Team Approach — 💡 Understand (K2)

Everyone on the team is responsible for quality — not just testers. Testers collaborate closely with developers and business representatives. This comes from Extreme Programming (XP).

Exception: in safety-critical systems, a higher level of test independence may be needed.

Independence of Testing — 💡 Understand (K2)

Independent testers find different defects than the author because they have different cognitive biases. Levels of independence: 1. Author tests their own work (lowest independence) 2. Peers from the same team 3.

Testers from outside the team (high independence) 4. Testers from outside the organization (very high independence)

Benefit: Different perspective catches more defects. **Drawback:** Risk of isolation, communication problems, developers losing sense of responsibility for quality, or creating an "us vs. them" mentality between testers and developers.

Key Terms Cheat Sheet — 🧠 Remember (K1)

Term	Definition
Error	A human mistake
Defect	A flaw introduced by an error
Failure	Incorrect behavior caused by executing a defect
Root cause	The fundamental reason a defect exists
Test basis	Everything used to derive test cases (requirements, designs, code, etc.)
Test object	The item being tested
Testware	All artifacts produced by test activities
Verification	Are we building the product right?
Validation	Are we building the right product?

Practice Questions

- What is the difference between error, defect, and failure? Give an example of each.
- List the 7 testing principles and explain any two in your own words.
- What is the difference between testing and QA?
- What is the difference between verification and validation?
- Why is exhaustive testing impossible?
- What are the two principal roles in testing and what does each focus on?
- What are the benefits and drawbacks of test independence?
- What does traceability help with in the test process?

Chapter 2 - Testing Throughout the Software Development Lifecycle

The big idea: Testing doesn't happen at the end. It lives inside every phase of development, adapts to whatever model the team uses, and changes shape depending on the context.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 2

LO	K	Topic
FL-2.1.1	K2	Explain the impact of the chosen SDLC model on testing
FL-2.1.2	K1	Recall good testing practices that apply to any SDLC model
FL-2.1.3	K1	Recall examples of TDD, ATDD, and BDD
FL-2.1.4	K2	Summarize the DevOps benefits for testing
FL-2.1.5	K2	Explain the shift-left approach
FL-2.1.6	K2	Explain how retrospectives can be used for process improvement in testing
FL-2.2.1	K2	Distinguish the different test levels
FL-2.2.2	K2	Distinguish the different test types
FL-2.2.3	K2	Distinguish confirmation testing from regression testing
FL-2.3.1	K2	Summarize the triggers for maintenance testing
FL-2.3.2	K2	Explain the role of impact analysis for maintenance testing

Exam tip: No K3 in this chapter. K1 items are FL-2.1.2 and FL-2.1.3 — just recall, no explanation needed. Everything else requires you to *explain* or *distinguish*.

2.1 Testing in the Context of an SDLC — 💡 Understand (K2)

An SDLC model defines how development phases and activities relate to each other. The choice of model directly impacts **how** testing is done — not whether testing is done.

The three main SDLC model categories — 💡 Understand (K2)

Model	Examples	How testing fits
Sequential	Waterfall, V-model	Testing begins after development; V-model maps each dev stage to a test level
Iterative	Spiral, Prototyping	Testing occurs within each repeated cycle
Incremental	Unified Process	Each small increment is tested before moving to the next

In practice, many real projects combine models — e.g., Agile uses both iterative and incremental approaches. Agile practices like Scrum, Kanban, TDD, BDD, and XP are methods that operate *within* these model types.

The SDLC impacts testing in 5 ways — 💡 Understand (K2)

1. Scope and timing of test activities (when and what gets tested)
2. Level of detail of test documentation
3. Choice of test techniques and approach
4. Extent of test automation
5. Roles and responsibilities of testers

Good testing practices — apply to ANY SDLC model — 🧠 Remember (K1)

- Every development activity has a corresponding test activity
- Different test levels have specific and different test objectives
- Test analysis and design begin during the corresponding development phase
- Testers review work products as soon as drafts are available

Testing as a Driver: TDD, ATDD, and BDD — 🧠 Remember (K1)

These approaches flip the traditional order — **tests are written before the code**.

Approach	Who uses it	How it works
TDD (Test-Driven Development)	Developers	Write a failing test → write code to pass it → refactor
ATDD (Acceptance Test-Driven Development)	Whole team	Derive tests from acceptance criteria before the feature is built
BDD (Behavior-Driven Development)	Whole team	Write tests in plain language using Given/When/Then format

All three implement the **early testing principle** and support **shift left**.

Exam tip (K1): For FL-2.1.3, you only need to recall who uses each approach and its core idea. The "how it works" details in the table are for context — the exam tests recognition, not reproduction of the procedure.

DevOps and Testing — 💡 Understand (K2)

DevOps bridges development, testing, and operations to deliver high-quality software faster.

Benefits for testing: - Fast feedback on code quality via CI/CD pipelines - Automated regression tests run on every commit - Visibility on non-functional quality (performance, reliability) increases - Reduces repetitive manual testing through automation

Exam scope: The *benefits* of DevOps for testing are directly tested (FL-2.1.4, K2). The risks below are supporting context not covered by a specific LO — understand them but don't memorize them.

Risks and challenges: - DevOps pipeline must be defined and maintained - Test automation requires additional resources - CI/CD tools must be introduced and kept up to date

Even with heavy automation, manual testing — especially from the user's perspective — is still needed.

Shift Left — 💡 Understand (K2)

Shift left means **testing earlier in the SDLC** — not waiting until the code is complete.

Good practices to achieve shift left: - Review specifications from a tester's perspective (find ambiguities early) - Write test cases before the code is written - Use CI/CD to get fast feedback on every code commit - Perform static analysis before dynamic testing - Start non-functional testing at the component level, where possible

Shift left may increase early effort and cost, but it saves significantly more later.

Retrospectives and Process Improvement — 💡 Understand (K2)

Retrospectives are held at the end of an iteration, project, or release milestone. Participants (not only testers — also developers, architects, product owners, business analysts) discuss: - What went well and should be retained? - What didn't go well and could be improved? - How to incorporate improvements going forward?

Benefits for testing: - Increased test effectiveness and efficiency - Improved quality of testware - Team bonding and learning - Better cooperation between development and testing

2.2 Test Levels and Test Types — 💡 Understand (K2)

Test Levels — the five levels — 💡 Understand (K2)

Each level has a different focus, test basis, and set of objectives.

Level	Focus	Typically performed by
Component testing (unit)	Individual components in isolation	Developers
Component integration testing	Interfaces and interactions between components	Developers / Testers
System testing	End-to-end behavior of the entire system	Independent testers
System integration testing	Interfaces between the system and external systems/services	Testers
Acceptance testing	Validation; demonstrating readiness for deployment	Business / End users

Acceptance Testing Types — 💡 Understand (K2)

- **UAT** (User Acceptance Testing): Real users validate the system meets their needs
- **OAT** (Operational Acceptance Testing): Operational readiness — backup, recovery, security
- **Contractual/Regulatory**: Verifies compliance with contracts or regulations
- **Alpha testing**: Internal users test at the developer's site
- **Beta testing**: External users test in their own environment

Test Types — what aspect is being tested — 💡 Understand (K2)

Type	What it checks
Functional testing	What the system does — functional completeness, correctness, appropriateness
Non-functional testing	How well the system behaves — performance, security, usability, reliability, maintainability, portability, safety
Black-box testing	Behavior based on specifications, without knowledge of internal structure
White-box testing	Internal structure — code paths, architecture, data flows

All four test types can be applied at **any** test level.

Note: "Black-box" and "white-box" appear in both Chapter 2 (as test *types*) and Chapter 4 (as test *technique categories*). In Chapter 2 they classify *what the tests are based on* (spec vs. structure); in Chapter 4 they classify *how test cases are derived*. Same terms, related but distinct meaning — don't let it confuse you.

Change-Related Testing — 💡 Understand (K2)

- **Confirmation testing**: Verifies that a specific defect has been fixed
- **Regression testing**: Ensures that changes haven't broken existing functionality

Regression testing grows with every release — it's a strong candidate for automation.

2.3 Maintenance Testing — 💡 Understand (K2)

Maintenance testing is triggered when a system that's already in production needs to change.

Triggers (v4 categories) — 💡 Understand (K2)

Trigger	Examples
Modifications	Planned enhancements, corrective changes (bug fixes), hot fixes
Upgrades or migrations	Moving to a new platform, OS, or environment
Retirement	End of life — data archiving and retrieval testing

Scope depends on — 💡 Understand (K2)

- Degree of risk of the change
- Size of the existing system
- Size of the change

Impact Analysis — 💡 Understand (K2)

Before testing, **impact analysis** determines which parts of the system could be affected by the change. This helps define the scope of regression testing and relies on good traceability.

Key Terms Cheat Sheet — 🧠 Remember (K1)

Term	Definition
SDLC	Software Development Lifecycle — the process to plan, create, test, and deliver software
Test level	A group of test activities organized together (component, integration, system, acceptance)
Test type	Classification by what is being tested (functional, non-functional, black-box, white-box)
Shift left	Testing earlier in the lifecycle to catch defects sooner
TDD	Tests are written before the code to drive development
BDD	Tests are written in Given/When/Then natural language format
Regression testing	Re-running tests to detect unintended side effects of a change
Confirmation testing	Re-testing after a fix to verify the defect is resolved
Impact analysis	Evaluating which parts of the system are affected by a change

Practice Questions

- What are the three main SDLC model categories? Give one example of each.
- How does the choice of SDLC model impact testing? Name at least three ways.

- What is the difference between TDD, ATDD, and BDD?
- What are the benefits and risks of DevOps for testing?
- What does "shift left" mean and how can you achieve it?
- What is the difference between component testing and component integration testing?
- What is the difference between confirmation testing and regression testing?
- What are the three triggers for maintenance testing in v4?
- Why is regression testing a good candidate for automation?
- What is the purpose of impact analysis in maintenance testing?

Chapter 3 - Static Testing

The big idea: You don't need to run software to find problems in it. Reviewing documents, designs, and code before execution often catches defects earlier, cheaper, and in places dynamic testing simply can't reach.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 3

LO	K	Topic
FL-3.1.1	K1	Recall work products that can be examined by static testing
FL-3.1.2	K2	Explain the value of static testing
FL-3.1.3	K2	Compare static testing and dynamic testing
FL-3.2.1	K1	Recall the activities, roles, and responsibilities of a formal review
FL-3.2.2	K2	Explain how the different review types differ from one another
FL-3.2.3	K1	Recall the factors for a successful review

Exam tip: No K3 in this chapter. Three K1 items (FL-3.1.1, FL-3.2.1, FL-3.2.3) — pure recall. FL-3.1.3 and FL-3.2.2 require you to *compare* and *explain differences* — classic K2.

3.1 Static Testing Basics — Understand (K2)

Static testing means evaluating a work product **without executing it**. It can be done: - **Manually** — through reviews (walkthroughs, inspections, etc.) - **With tools** — through static analysis (e.g., linters, code analyzers integrated into CI pipelines)

It applies to both **verification** and **validation**, and its objectives include improving quality, detecting defects, and assessing readability, completeness, correctness, testability, and consistency.

What Can Be Examined with Static Testing? — Remember (K1)

Almost **any work product** — as long as it can be read and understood:

- Requirement specification documents

- Source code
- Test plans and test cases
- Product backlog items, test charters
- Project documentation, contracts, and models

For static analysis specifically, the work product needs a formal syntax (e.g., code, models, or structured text).

NOT appropriate for static testing: work products that are difficult for humans to interpret or that can't be analyzed by tools for legal reasons (e.g., 3rd party executable code).

Why Bother? The Value of Static Testing — 💡 Understand (K2)

Static testing is powerful because:

1. **It catches defects early** — before they get baked into later work products and become expensive to fix
2. **It finds things dynamic testing can't** — like unreachable code, design flaws, or defects in non-executable documents
3. **It builds shared understanding** — reviewing requirements together aligns the team early
4. **It's cost-effective** — the upfront cost of reviews is far lower than the cost of finding those same defects late in the project

Defects that are easier and cheaper to find through static testing — 💡 Understand (K2)

Category	Examples
Requirements defects	Inconsistencies, ambiguities, contradictions, omissions, duplications
Design defects	Inefficient database structures, poor modularization
Coding defects	Undefined values, undeclared variables, unreachable/duplicated code, excessive complexity
Standards deviations	Naming convention violations, coding standard violations
Interface defects	Mismatched number, type, or order of parameters
Security vulnerabilities	Buffer overflows
Test basis gaps	Missing tests for an acceptance criterion

Static vs. Dynamic Testing — Side by Side — 💡 Understand (K2)

Aspect	Static Testing	Dynamic Testing
Needs execution?	No	Yes
How defects are found	Directly	Via failures → root cause analysis
Applicable to	Executable AND non-executable work products	Executable work products only
Rare code paths	Easier to detect	Hard to reach
Quality characteristics	Maintainability, readability	Performance, reliability

Think of it this way: **static testing reads the recipe**, while **dynamic testing tastes the dish**.

3.2 Feedback and Review Process — 💡 Understand (K2)

Why Early and Frequent Stakeholder Feedback Matters — 💡 Understand (K2)

If stakeholders aren't involved regularly during development, the product may drift from their original vision. The consequences: - Costly rework - Missed deadlines - Blame games - Project failure

Frequent feedback throughout the SDLC: - Prevents misunderstandings about requirements early - Ensures changes are understood and implemented before they become expensive - Keeps the team focused on what delivers the most value - Addresses risks proactively

The Review Process — 5 Activities — 🧠 Remember (K1)

Based on the **ISO/IEC 20246** standard:

Step	What happens
1. Planning	Define scope, purpose, exit criteria, effort, and timeframes
2. Review initiation	Ensure all participants have access, understand their role, and are prepared
3. Individual review	Each reviewer independently assesses the work product; logs anomalies, recommendations, and questions
4. Communication and analysis	Anomalies are discussed in a review meeting; decisions made on status, ownership, and follow-up actions
5. Fixing and reporting	The author fixes the defects found; the review leader writes the review report; the work product is accepted once exit criteria are met

Roles in a Review — Who Does What? — 🧠 Remember (K1)

Role	Responsibility
Manager	Decides what gets reviewed; provides resources (people, time)
Author	Creates and later fixes the work product under review
Moderator (facilitator)	Runs the review meeting; handles mediation, time, and ensures a safe environment
Scribe (recorder)	Records anomalies, decisions, and new findings during the meeting
Reviewer	Performs the actual review; can be a team member, SME, or any stakeholder
Review leader	Owens the review overall; decides who participates, when, and where

Memory tip: Think of a review as a structured meeting with a **boss** (manager), a **creator** (author), a **referee** (moderator), a **note-taker** (scribe), **evaluators** (reviewers), and an **organizer** (review leader).

Review Types — From Least to Most Formal — 💡 Understand (K2)

Type	Formality	Led by	Key characteristics
Informal review	Lowest	Anyone	No defined process; no formal output; just find anomalies
Walkthrough	Low–Medium	Author	Author guides the team through the work product; educates, gains consensus, detects anomalies; individual review beforehand is optional; scribe is mandatory
Technical review	Medium–High	Trained moderator or peer leader	Technically qualified reviewers; focus on consensus on technical problems and detecting anomalies
Inspection	Highest	Moderator (NOT the author)	Follows the full defined process; maximum anomaly detection is the main goal; metrics are collected; author cannot be review leader or scribe

Key exam point: In an inspection, the **author cannot act as review leader or scribe** — this is unique to this review type.

Success Factors for Reviews — 🧠 Remember (K1)

A review succeeds when:

- Objectives are **clear and measurable** — evaluation of participants is **never** an objective
- The **right review type** is chosen for the work product and context
- Reviews are done on **small chunks** so reviewers stay focused
- Feedback** is given to stakeholders and authors so they can improve

- Participants have **adequate preparation time**
- **Management supports** the review process
- Reviews are part of the **organization's culture**
- All participants receive **adequate training**
- Meetings are **facilitated effectively**

Key Terms Cheat Sheet — 🧠 Remember (K1)

Term	Definition
Static testing	Evaluating a work product without executing it
Dynamic testing	Testing by executing the software
Static analysis	Tool-assisted evaluation against a formal syntax
Review	Manual evaluation of a work product by one or more people
Inspection	Most formal review type; follows a complete defined process; author cannot be leader or scribe
Technical review	Review led by a moderator with technically qualified reviewers
Walkthrough	Review led by the author
Informal review	Review with no defined process or formal output
Anomaly	Any result that differs from what is expected; identified during a review
Moderator	Ensures the effective running of a review meeting

Practice Questions

- What is static testing and how does it differ from dynamic testing?
- What types of work products can be examined by static testing?
- Name three defect types that are easier to find through static testing.
- What are the five activities in the review process?
- What are the six principal roles in a review? What does each one do?
- What is the most formal review type and what makes it unique?
- What is the main difference between a walkthrough and a technical review?
- Why can't the author be the review leader or scribe in an inspection?
- List four success factors for reviews.
- Why should evaluation of participants never be an objective of a review?

Chapter 4 - Test Analysis and Design

The big idea: How do you decide *what* to test and *how* to test it without just guessing? Test techniques give you a systematic, repeatable way to design test cases that cover what matters — based on specifications, code structure, or your own experience.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 4

LO	K	Topic
FL-4.1.1	K2	Summarize the categories of test techniques and their subcategories
FL-4.2.1	K3	Apply equivalence partitioning to derive test cases for a given scenario
FL-4.2.2	K3	Apply boundary value analysis to derive test cases for a given scenario
FL-4.2.3	K3	Apply decision table testing to derive test cases for a given scenario
FL-4.2.4	K3	Apply state transition testing to derive test cases for a given scenario
FL-4.3.1	K2	Explain statement testing
FL-4.3.2	K2	Explain branch testing
FL-4.3.3	K2	Explain the value of white-box testing
FL-4.4.1	K2	Explain error guessing
FL-4.4.2	K2	Explain exploratory testing
FL-4.4.3	K2	Explain checklist-based testing
FL-4.5.1	K2	Explain the factors that make a good user story (INVEST criteria)
FL-4.5.2	K2	Recall the typical formats for writing acceptance criteria
FL-4.5.3	K2	Summarize the activities of the ATDD approach

Exam tip: The four black-box techniques (FL-4.2.1–4.2.4) are **K3** — the exam will give you a scenario and ask you to produce test cases. Practice deriving them, not just explaining them. White-box techniques (FL-4.3.1–4.3.2) are only K2 — you explain statement and branch coverage, not calculate it.

4.1 The Three Categories of Test Techniques — Understand (K2)

Think of test techniques as three different lenses for looking at a system:

Category	Also known as	Based on	Key characteristic
Black-box	Specification-based	Specified behavior	Tests are independent of implementation — they survive code changes
White-box	Structure-based	Internal structure/code	Tests depend on how the software is built; created after design/implementation
Experience-based	—	Tester's knowledge and skill	Complementary to the other two; finds defects the others miss

Memory tip: Black = you can't see inside the box. White = you can see everything inside. Experience = you've seen similar boxes break before.

4.2 Black-Box Test Techniques — Apply (K3)

4.2.1 Equivalence Partitioning (EP) — Apply (K3)

The idea: If the system treats all values in a group the same way, you only need to test one value from that group.

- Divide input (or output) data into **partitions** where all elements are processed identically
- One test case per partition is sufficient
- **Valid partition:** contains values the system should accept
- **Invalid partition:** contains values the system should reject
- **100% EP coverage** = exercise every partition (valid AND invalid) at least once
- Coverage = partitions exercised ÷ total partitions

When multiple inputs exist: Use **Each Choice coverage** — at least one value from each partition set appears in at least one test case.

Example: A field accepts ages 18–65. Partitions: under 18 (invalid), 18–65 (valid), over 65 (invalid). You need at least 3 test cases — one per partition.

4.2.2 Boundary Value Analysis (BVA) — Apply (K3)

The idea: Developers make the most mistakes at the edges of a range — so test there.

- Extends EP by specifically targeting the **boundaries** of ordered partitions
- Only applicable to **ordered** partitions (where min/max values exist)

Two versions:

Version	Coverage items per boundary	More rigorous?
2-value BVA	Boundary value + its closest neighbor from the adjacent partition	Less
3-value BVA	The boundary value itself, the value immediately below it, and the value immediately above it	More — can catch off-by-one errors 2-value misses

Example: For a range 1–10 using 3-value BVA, test: 0, 1, 2 (lower boundary) and 9, 10, 11 (upper boundary).

100% coverage: Exercise all boundary values (2-value) or all boundary values and their neighbors (3-value).

4.2.3 Decision Table Testing — ⚙️ Apply (K3)

The idea: When the system behaves differently depending on combinations of conditions, a table maps every combination to its expected outcome.

- Best for testing **business rules** and complex logic
- Each **column** = one decision rule (a unique combination of conditions + resulting actions)
- Notation:
- **T** = condition is true, **F** = condition is false
- **–** = condition is irrelevant to the outcome
- **N/A** = condition is infeasible for this rule
- **X** = action occurs, **blank** = action does not occur

100% coverage = exercise all columns with feasible condition combinations. Coverage = feasible columns exercised ÷ total feasible columns.

Note: Coverage is measured against the actual columns in the table being used. If the table has been minimized (rules collapsed), coverage is calculated against its actual columns — not against the theoretical 2^n maximum.

Strength: Reveals gaps and contradictions in requirements; systematic — nothing gets overlooked.

Weakness: With many conditions, the number of rules grows exponentially (2^n). Use a minimized table or risk-based approach to reduce scope.

4.2.4 State Transition Testing — ⚙️ Apply (K3)

The idea: Some systems behave differently depending on their current state. This technique tests those state-dependent behaviors.

Key concepts: - **State diagram:** shows all possible states and the valid transitions between them - Transition syntax: **event** [guard condition] / **action** - **State table:** equivalent to a state diagram; rows = states, columns = events; empty cells = invalid transitions

A test case = a sequence of events that produces a sequence of state changes.

Three coverage criteria (weakest → strongest):

Coverage	What it exercises	Notes
All states	All states at least once	Weakest — can be achieved without testing all transitions
Valid transitions (0-switch)	All valid transitions — each valid transition exercised at least once, without chaining sequences (0-switch = single steps only)	Most widely used criterion
All transitions	All valid + all invalid transitions	Strongest; required for mission/safety-critical software

Key exam point: Achieving valid transitions coverage guarantees all states coverage. Achieving all transitions coverage guarantees both. Test only **one** invalid transition per test case to avoid defect masking.

4.3 White-Box Test Techniques — 💡 Understand (K2)

4.3.1 Statement Testing — 💡 Understand (K2)

- **Coverage items:** executable statements
- Aim: design test cases to execute statements until an acceptable coverage level is reached
- **100% statement coverage** = every executable statement run at least once
- Coverage = statements exercised ÷ total executable statements

Limitation: 100% statement coverage does NOT guarantee all branches are tested. A defect that only triggers under a specific condition may be missed.

4.3.2 Branch Testing — 💡 Understand (K2)

- **Coverage items:** branches — transfers of control between nodes in a control flow graph
- Branches can be **unconditional** (straight-line code) or **conditional** (if/else, switch, loop decisions)
- **100% branch coverage** = all branches exercised
- Coverage = branches exercised ÷ total branches

Critical rule: Branch coverage subsumes statement coverage. If you achieve 100% branch coverage, you automatically achieve 100% statement coverage. The reverse is NOT true.

Limitation: Doesn't detect defects that only appear on a specific path through the code.

4.3.3 The Value of White-Box Testing — 💡 Understand (K2)

Strength: Takes the entire implementation into account — can find defects even when the spec is vague, outdated, or incomplete.

Weakness: If the software is missing a required feature entirely (defect of omission), white-box testing won't detect it — it only tests what's there.

Also useful in static analysis: White-box *analysis* (a form of static testing) can be applied to pseudocode or unexecutable code to review structure without running it. White-box *testing* (dynamic) requires executable code — don't confuse the two.

Black-box testing alone gives you no measure of actual code coverage. White-box testing provides that objective measurement.

4.4 Experience-Based Test Techniques — 💡 Understand (K2)

4.4.1 Error Guessing — 💡 Understand (K2)

The idea: Use what you know about how software breaks to anticipate where it will break.

Based on the tester's knowledge of: - How the application has worked (or failed) in the past - Types of mistakes developers typically make - Types of failures common in similar applications

Fault attacks: Create a structured list of likely errors, defects, and failures → design tests specifically to expose them. Lists can come from experience, defect data, or common failure knowledge.

4.4.2 Exploratory Testing — 💡 Understand (K2)

The idea: Design, execute, and evaluate tests simultaneously while learning about the system.

- Testing is used to learn, explore deeper, and find untested areas
- **Session-based approach:** testing within a defined time box, guided by a **test charter** containing test objectives
- Each session ends with a **debriefing** with stakeholders

Best used when: - Specifications are few, inadequate, or missing - There's significant time pressure - You want to complement more formal techniques

More effective when the tester: is experienced, has domain knowledge, and brings analytical thinking, curiosity, and creativity.

4.4.3 Checklist-Based Testing — 💡 Understand (K2)

The idea: Use a prepared checklist of test conditions to guide testing — especially when detailed test cases don't exist.

- Items are often phrased as questions
- Built from experience, user knowledge, and understanding of how software fails
- Should be **regularly updated** based on defect analysis — checklists can wear out just like tests

Do NOT include in checklists (*industry guidance — not explicitly listed in the CTFL 4.0 syllabus*): - Items that can be checked automatically - Items better suited as entry/exit criteria - Items that are too general

Trade-off: High-level checklists provide consistency and coverage breadth, but less repeatability than detailed test cases.

4.5 Collaboration-Based Test Approaches — 💡 Understand (K2)

Unlike the other techniques (which focus on *finding* defects), collaboration-based approaches focus on **avoiding defects through shared understanding**.

4.5.1 Collaborative User Story Writing — 💡 Understand (K2)

A **user story** represents a feature valuable to a user or purchaser. Every user story has **3 Cs** (Jeffries 2000):

C	What it is
Card	The medium describing the story (index card, electronic board entry)
Conversation	Explains how the software will be used (verbal or documented)
Confirmation	The acceptance criteria

Standard format: *"As a [role], I want [goal], so that I can [resulting business value]."*

Good user stories follow INVEST: - Independent - Negotiable - Valuable - Estimable - Small - Testable

If a stakeholder can't imagine how to test a user story, it's a signal the story is unclear or not valuable enough.

4.5.2 Acceptance Criteria — 💡 Understand (K2)

Acceptance criteria are the **conditions a user story must meet to be accepted** by stakeholders. They are essentially the test conditions for acceptance testing.

Used to: - Define the scope of the user story - Reach consensus among stakeholders - Describe both positive and negative scenarios - Serve as the basis for acceptance tests - Allow accurate planning and estimation

Two most common formats:

Format	Example
Scenario-oriented (BDD)	Given [context], When [action], Then [outcome]
Rule-oriented	Bullet point verification list or input-output mapping table

4.5.3 Acceptance Test-Driven Development (ATDD) — 💡 Understand (K2)

The idea: Write test cases *before* implementing the user story, using the acceptance criteria as the guide.

How it works: 1. **Specification workshop:** team (customers + developers + testers) analyzes the user story and acceptance criteria together; resolves ambiguities and gaps 2. **Test case creation:** test cases are created based on acceptance criteria; they serve as examples of correct behavior 3. **Implementation:** developers write code to pass the test cases

Order of test cases: positive (happy path) → negative → non-functional quality characteristics (e.g., performance, usability)

Test cases must be expressed in language **understandable by stakeholders** (natural language: preconditions, inputs, postconditions).

Key Terms Cheat Sheet — 🧠 Remember (K1)

Term	Definition
Black-box technique	Derives tests from specified behavior; independent of internal structure
White-box technique	Derives tests from internal structure/code
Experience-based technique	Uses tester's knowledge and experience to design tests
Equivalence partitioning	Divides data into partitions processed identically; one test per partition
Boundary value analysis	Tests at the edges of equivalence partitions
Decision table testing	Tests combinations of conditions and their resulting actions
State transition testing	Tests behavior based on system states and transitions between them
Statement coverage	% of executable statements exercised by tests
Branch coverage	% of branches exercised; subsumes statement coverage
Error guessing	Anticipating defects based on experience and failure knowledge
Exploratory testing	Simultaneous design, execution, and evaluation within a time-boxed session
Checklist-based testing	Testing guided by a prepared list of conditions, often as questions
User story	A description of a feature valuable to a user; has Card, Conversation, Confirmation
INVEST	Criteria for a good user story: Independent, Negotiable, Valuable, Estimable, Small, Testable
Acceptance criteria	Conditions a user story must meet to be accepted; test conditions for acceptance tests
ATDD	Test cases written before the user story is implemented, based on acceptance criteria

Practice Questions

- What are the three categories of test techniques and how do they differ?
- In equivalence partitioning, why is one test per partition considered sufficient?
- What is the difference between 2-value BVA and 3-value BVA? When would you use each?
- What does each symbol mean in a decision table (T, F, –, N/A, X)?
- What are the three state transition coverage criteria, from weakest to strongest?
- Why should you test only one invalid transition per test case in state transition testing?
- What does "branch coverage subsumes statement coverage" mean?

- What is the main strength and weakness of white-box testing?
- What is a fault attack in error guessing?
- When is exploratory testing most useful?
- What are the 3 Cs of a user story?
- What does INVEST stand for?
- What are the two most common formats for writing acceptance criteria?
- What happens in the specification workshop in ATDD?
- What is the main difference between defect detection techniques (4.2–4.4) and collaboration-based approaches (4.5)?

Chapter 5 - Managing Test Activities

The big idea: Good testing doesn't just happen — it has to be planned, tracked, and wrapped up properly. This chapter is about the organizational and managerial side of testing: how to define strategy, estimate effort, measure progress, manage risk, and report results.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 5

LO	K	Topic
FL-5.1.1	K2	Exemplify the purpose and content of a test plan
FL-5.1.2	K1	Recognize how a tester adds value to iteration and release planning
FL-5.1.3	K2	Compare entry criteria and exit criteria
FL-5.1.4	K3	Apply estimation techniques to calculate test effort for a defined scenario
FL-5.1.5	K3	Apply test case prioritization for a given scenario
FL-5.1.6	K2	Recall the concepts of the test pyramid
FL-5.1.7	K2	Summarize the testing quadrants and their relationship to test levels and test types
FL-5.2.1	K2	Explain the purpose of test monitoring and control and the difference between them
FL-5.2.2	K2	Summarize the metrics used for testing
FL-5.2.3	K2	Summarize the purpose and content of test reports
FL-5.3.1	K2	Explain how risks influence the scope of testing
FL-5.3.2	K1	Recall risk identification and risk assessment
FL-5.4.1	K3	Apply a defect report for a given scenario
FL-5.4.2	K2	Summarize the content and purpose of a defect report

Exam tip: Three K3 items here — estimation (FL-5.1.4), prioritization (FL-5.1.5), and defect reports (FL-5.4.1). The exam will give you a situation and ask you to produce something: an estimate, a prioritized list, or a defect report. FL-5.1.2 and FL-5.3.2 are K1 — pure recall.

5.1 Test Planning — 💡 Understand (K2)

A **test plan** documents the means and schedule for achieving test objectives. It's a living document — updated as the project evolves.

What a test plan typically includes — 💡 Understand (K2)

Section	Content
Context	Scope, test basis, constraints, test environment
Objectives	What testing aims to achieve
Stakeholders	Roles, responsibilities, communication
Test approach	Strategy, techniques, types of testing
Schedule	Milestones, effort, timelines
Resources	People, tools, environments
Entry and exit criteria	When to start and when to stop testing
Risk and issues	Known risks and mitigation strategies
Metrics	What will be measured to track progress

5.2 Tester's Contribution to Iteration and Release Planning — 🧠 Remember (K1)

In Agile and iterative contexts, testers actively participate in:

- **Release planning** — estimating effort for features across multiple iterations; analyzing risks; defining the overall test approach
- **Iteration planning** — analyzing user stories in the backlog; assessing testability; identifying acceptance criteria; estimating test effort per story; participating in risk identification and assessment

Testers bring a quality perspective that developers and product owners may not naturally have — challenging assumptions, spotting missing acceptance criteria, and flagging ambiguities early.

5.3 Entry Criteria and Exit Criteria — 💡 Understand (K2)

Entry criteria — when to START testing — 💡 Understand (K2)

Conditions that must be met before a test activity begins. Testing without these satisfied wastes effort.

Examples: - Testware is available (test cases, test scripts, test data) - Test environment is ready and configured - The test object has passed a smoke test

Exit criteria — when to STOP testing — 💡 Understand (K2)

Conditions that determine when a test activity is sufficiently complete.

Examples: - Planned tests have been executed - Coverage level achieved (statements, requirements) - Defect density is below the defined threshold - Time or budget has been exhausted (pragmatic exit)

In Agile, entry/exit criteria are often called **Definition of Ready (DoR)** and **Definition of Done (DoD)**.

5.4 Estimation Techniques — ⚙️ Apply (K3)

Estimating how much testing effort is needed is notoriously difficult. ISTQB covers two main techniques:

Metrics-Based Estimation — ⚙️ Apply (K3)

Use data from previous similar projects to estimate current effort.

- Derive estimates from actual metrics: defect rates, test execution times, number of test cases per function point
- Useful when historical data is available and the current project is similar enough

Expert-Based Estimation — ⚙️ Apply (K3)

Use the knowledge and judgment of experienced testers.

Common approaches: - **Planning Poker:** team members simultaneously reveal effort estimates; outliers are discussed until consensus is reached (used in Agile) - **Wideband Delphi:** structured rounds of anonymous estimation and discussion until the team converges (*from older ISTQB versions — CTFL 4.0 only names Planning Poker as an example of expert-based estimation*)

Both techniques can be combined — use historical data as a baseline and expert judgment to adjust for known differences.

5.5 Test Case Prioritization — ⚙️ Apply (K3)

Not all tests are equally important. Prioritization determines the order in which tests are run.

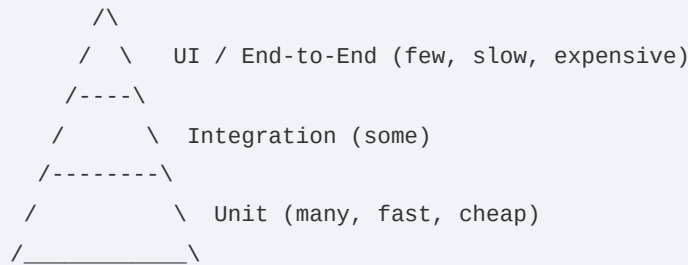
Strategies — ⚙️ Apply (K3)

Strategy	Prioritize by
Risk-based	Highest risk areas first — most impact if they fail
Coverage-based	Tests that cover more requirements or code first
Requirements-based	Tests linked to the most critical or important requirements
Activity-based	Dependency-driven order (some tests must run before others) (<i>supporting concept — not defined in the CTFL 4.0 syllabus</i>)

In practice, risk-based prioritization is the most common and the most effective — it maximizes defect detection in the shortest time when time is limited.

5.6 The Test Pyramid — 💡 Understand (K2)

The **test pyramid** is a model that groups tests by granularity and recommends how to distribute them:



- **Bottom (unit):** many small, fast, isolated tests — cheap to write and maintain
- **Middle (integration):** tests of interactions between components
- **Top (UI/E2E):** few, slow, brittle tests — use only where needed

The pyramid implies: **automate at the lowest level possible**. The more you push automation up to the UI level, the slower and more fragile your test suite becomes.

5.7 Testing Quadrants — 💡 Understand (K2)

The **testing quadrants** (Brian Marick) categorize test types by two axes:

- **Business-facing vs. Technology-facing**
- **Supporting the team (guiding development) vs. Critiquing the product (finding problems)**

	Supporting the team	Critiquing the product
Technology-facing	Q1: Component tests, unit tests, integration tests	Q4: Performance, security, reliability, "ility" tests
Business-facing	Q2: Automated functional tests, story tests, prototypes, simulations	Q3: Exploratory testing, usability testing, UAT, alpha/beta

The quadrants are a **communication tool** — they help teams talk about what kinds of testing are needed and who should do them. They are not prescriptive.

5.8 Risk and Testing — 💡 Understand (K2)

Risk is central to test planning. It drives prioritization, coverage decisions, and the depth of testing applied to each area.

Key concepts — 🧠 Remember (K1)

- **Risk**: the chance of an event occurring that causes an adverse outcome
- **Risk level**: a combination of **likelihood** (probability) and **impact** (consequence)
- **Product risk**: risk that something in the system will fail — drives testing scope
- **Project risk**: risk to the project itself (schedule, budget, team) — managed through project management

Risk-Based Testing — 💡 Understand (K2)

Uses risk as the primary guide for: - **What** to test (high-risk areas get more focus) - **How much** to test (riskier areas get more test cases) - **When** to test (high-risk items are tested earlier)

Two stages: 1. **Risk identification**: brainstorm with stakeholders — what could go wrong? 2. **Risk assessment**: evaluate likelihood and impact; prioritize accordingly

If a high-risk area receives adequate testing and no defects are found, this provides evidence of quality — not just the absence of failures.

5.9 Test Monitoring, Test Control, and Test Completion — 💡 Understand (K2)

Test Monitoring — 💡 Understand (K2)

Ongoing activity of checking test progress against the plan.

Common test metrics collected:

Metric	What it tells you
% of test cases executed	How much of the plan is done
% passed / failed / blocked	Quality signal
Defect detection rate	How many defects are being found over time
Defect density by module	Where defects cluster
Requirements / code coverage	How thoroughly the system is exercised
Resource / budget consumption	Is the project on track?

Test Control — 💡 Understand (K2)

Taking corrective actions when monitoring reveals deviations from the plan.

Examples of control actions: - Reprioritize tests when a critical defect is found - Adjust schedule or scope if risks materialize - Change the test approach (e.g., add exploratory sessions) - Add resources if the team is falling behind

Monitoring without control is just observation. Control is what makes monitoring useful.

Test Progress Reports — 💡 Understand (K2)

Communicate current status to stakeholders. Typically include: - Test period covered - Progress against the plan - Impediments and blockers - Planned activities for the next period

Test Completion Reports — 💡 Understand (K2)

Produced at the end of a test phase, iteration, or project. Summarize: - What was tested, how, and with what results - Deviations from the plan - Defects found and their status - Testware to archive or hand off - Lessons learned

5.10 Configuration Management in Testing — 💡 Understand (K2)

Exam scope: CM is supporting context for FL-5.1.1 (test plan content) — not a standalone LO in CTFL 4.0. Understand its purpose and how it relates to testware traceability, but don't memorize this section in depth.

Configuration management (CM) ensures that testware remains consistent, identifiable, and traceable.

What CM does for testing: - Identifies and tracks versions of testware (test cases, scripts, environments, data) - Ensures test cases link to the correct version of the test object - Enables reproducibility — running the same test in the same environment - Supports change control — if the software changes, what tests need to be updated?

In practice: your test cases should reference specific software versions. "Tested with v2.3.1" is meaningful only if CM ensures that v2.3.1 can be reproduced.

5.11 Defect Management — ⚙️ Apply (K3)

The defect management process tracks defects from discovery to closure.

Defect lifecycle (typical) — 💡 Understand (K2)

Note: The CTFL 4.0 syllabus does not prescribe specific lifecycle states. The diagram below is a common industry pattern for illustration. For the exam, focus on the **defect report fields** (FL-5.4.1/5.4.2), not the lifecycle state names.

New → Assigned → Open → Fixed → Retest → Closed
↘ Rejected / Deferred

Field	Purpose
Unique ID	Tracking
Summary	Short description of the failure
Date and author	Who found it and when
Environment	OS, browser, version, configuration
Steps to reproduce	Exact sequence to reproduce the failure
Expected result	What should have happened
Actual result	What actually happened
Severity	Impact on the system (Critical, Major, Minor, Trivial)
Priority	How urgently it needs to be fixed
Status	Current state in the lifecycle
Attachments	Logs, screenshots, videos

Severity vs. Priority: A typo in a login button label is low severity but possibly high priority (it's the first thing every user sees). A crash in a rarely-used feature might be high severity but low priority.

Term	Definition
Test plan	Document describing the scope, approach, schedule, and resources for testing
Entry criteria	Conditions that must be met before a test activity begins
Exit criteria	Conditions that determine when a test activity is complete
Risk	The possibility of an event with an adverse consequence
Product risk	Risk that a system component fails to meet expected behavior
Project risk	Risk to the project schedule, budget, or resources
Risk-based testing	Using risk as the guide for prioritizing what, how much, and when to test
Test pyramid	Model recommending more unit tests than integration or UI tests
Testing quadrants	Framework categorizing tests by audience (business vs. technology) and purpose (guiding vs. critiquing)
Test monitoring	Checking test progress against the plan
Test control	Taking corrective action when deviations are found
Defect report	A document capturing a failure and all information needed to reproduce, track, and resolve it
Severity	The impact of a defect on the system
Priority	The urgency with which a defect should be fixed
Configuration management	Tracking versions of testware to ensure consistency and reproducibility

Practice Questions

- What is the purpose of a test plan and what does it typically include?
- What is the difference between entry criteria and exit criteria? Give one example of each.
- What are the two main estimation techniques covered in ISTQB? How do they differ?
- What is risk-based test prioritization and why is it the most common strategy?
- What is the test pyramid and what does it recommend about automation?
- What are the four testing quadrants and what axes define them?
- What is the difference between product risk and project risk?
- What are the two stages of risk-based testing (identification and assessment)?
- What is the difference between test monitoring and test control?
- What should a test completion report include?
- What is the difference between severity and priority in defect management?
- Why does configuration management matter for testing?
- How do entry/exit criteria relate to Definition of Ready and Definition of Done in Agile?

- What metrics would you collect to monitor test progress?

Chapter 6 - Test Tools

The big idea: The right tool makes testing faster, more consistent, and more thorough — but tools are not a silver bullet. Knowing what categories of tools exist, what they actually do, and how to choose them wisely is what this chapter is about.

Cognitive Levels in This Chapter

K1 = Remember · K2 = Understand · K3 = Apply — see Chapter 1 for the full legend.

Learning Objectives for Chapter 6

LO	K	Topic
FL-6.1.1	K2	Explain how different types of test tools support testing
FL-6.2.1	K1	Recall the benefits and risks of test automation
FL-6.2.2	K1	Recall the considerations for selecting a tool

Exam tip: Chapter 6 is the shortest chapter and has **no K3**. The highest level is K2 (FL-6.1.1). FL-6.2.1 and FL-6.2.2 are K1 — recall, not explanation. Don't over-invest time here.

6.1 Tool Support for Testing — Understand (K2)

Test tools can support almost any test activity. They are categorized by the **activity they support**, not by what they are called — many modern tools cover multiple categories.

Key exam point (FL-6.1.1): You need to be able to match a tool category to the test activity it supports. The question format is typically: "which type of tool would support activity X?"

Management and Reporting Tools — 💡 Understand (K2)

Tool type	What it supports
Test management tools	Planning, monitoring, and controlling test activities; managing test cases, test suites, and execution schedules
Defect tracking tools	Logging, tracking, and managing defects through their lifecycle
Configuration management tools	Tracking versions of the software under test and testware; ensuring reproducibility
Requirements management tools	Traceability between requirements and test cases

Static Testing Tools — 💡 Understand (K2)

Tool type	What it supports
Static analysis tools	Analyzing source code without executing it — detecting coding standard violations, security vulnerabilities, unreachable code, undefined variables
Spelling and grammar checkers	Reviewing written work products (requirements, test cases) for language quality

Static analysis tools are part of static testing (Chapter 3). They complement manual reviews and can be integrated into CI/CD pipelines to give developers immediate feedback.

Test Design and Implementation Tools — 💡 Understand (K2)

Tool type	What it supports
Test data generation tools	Creating input data for tests automatically — useful for boundary values, large datasets, and masked production data
Test case design tools	Generating test cases from models or specifications (used in model-based testing)
Model-based testing tools	Automatically deriving test cases from a behavioral model of the system

Test Execution and Coverage Tools — 💡 Understand (K2)

Tool type	What it supports
Test execution tools (test automation frameworks)	Running test scripts automatically; comparing actual vs. expected results; logging results
Coverage measurement tools	Measuring how much of the code or requirements has been exercised by tests (e.g., statement coverage, branch coverage)

Common test execution approaches:

- **Data-driven testing:** the test script is separated from the test data; data is provided from external sources (spreadsheet, database). Same script runs with many data sets.
- **Keyword-driven testing:** test logic is expressed as readable keywords (e.g., `Login` , `ClickButton`); less technical for non-programmers to maintain.

Non-Functional Testing Tools — 💡 Understand (K2)

Tool type	What it supports
Performance testing tools	Simulating load, measuring response times, identifying bottlenecks (load testing, stress testing, endurance testing)
Security testing tools	Scanning for vulnerabilities, testing authentication and authorization, penetration testing support
Accessibility testing tools	Verifying that the system can be used by people with disabilities

DevOps and Collaboration Tools — 💡 Understand (K2)

Tool type	What it supports
CI/CD pipeline tools	Automatically triggering test execution on every code commit; integrating testing into the build process
Version control / SCM tools	Managing source code and testware versions; enabling branching and parallel development
Collaboration tools	Communication, shared test documentation, remote team coordination

DevOps tools are what make continuous testing possible — they connect the developer's commit to the test execution result without manual intervention (Chapter 2 connection: shift left, DevOps).

6.2 Benefits and Risks of Test Automation — 🧠 Remember (K1)

Test automation means using tools to execute tests and compare actual results to expected results automatically. It is most valuable for **repetitive, stable, high-volume testing** — not for everything.

Benefits of Test Automation — 🧠 Remember (K1)

Benefit	Why it matters
Reduces repetitive manual work	Frees testers to focus on exploratory and higher-value testing
Prevents simple human errors	A script executes exactly the same way every time
Objective assessment	Coverage metrics and results are measured consistently
Easy access to test status info	Results dashboards and logs are generated automatically
Reduced test execution time	Tests can run in parallel, overnight, or on every commit
More testing in the same time	Regression suites that would take days manually can run in hours

Risks of Test Automation — 🧠 Remember (K1)

Risk	What can go wrong
Unrealistic expectations	Tools don't find bugs automatically — they only check what they are told to check
Underestimated effort	Automating tests takes significant time to set up, script, and maintain
Using the wrong tool	Not every tool fits every technology stack or test type
Over-reliance on automation	Automated tests can't replace human judgment, especially for UX and exploratory testing
Maintenance burden	When the system changes, test scripts break and must be updated
False sense of security	Passing automated tests does not mean the system is defect-free

The key insight: Automation is a multiplier — it multiplies the efficiency of good testing, but it also multiplies the problems of bad testing. Bad tests automated at scale are worse than no automation.

6.3 Considerations for Selecting a Test Tool — 🧠 Remember (K1)

Choosing a tool is not just a technical decision — it depends on the organization, the project, and the team.

Factors to consider — 🧠 Remember (K1)

Factor	What to evaluate
Organizational maturity	Does the team have the skills to use and maintain the tool?
Technology of the SUT	Is the tool compatible with the system's tech stack (language, platform, browser)?
Gaps in the test process	What activities currently lack tool support? Which would benefit most?
Licensing and cost	Open source vs. commercial; one-time vs. subscription; total cost of ownership
Vendor support	Is training available? Is the vendor active and responsive?
Integration with existing tools	Does it fit into the current CI/CD pipeline, test management system, and defect tracker?

Good practices for tool selection — 🧠 Remember (K1)

- Run a **proof of concept**: try the tool on a real part of the project before committing
- Evaluate the **vendor's responsiveness** and the community around the tool
- Assess **training requirements** — factor in ramp-up time
- Involve **both testers and developers** in the selection decision
- Prefer tools that **integrate well** over tools that require everything to be rebuilt around them

Important distinction: Selecting a tool is not the same as implementing test automation. Tool selection is the decision; implementation is the project that follows — with its own planning, risks, and effort estimation.

Term	Definition
Test automation	Using tools to execute tests and compare actual results to expected results automatically
Test automation framework	A set of rules, libraries, and tools that provide the structure for automated test scripts
Data-driven testing	A test automation approach where test data is stored externally and fed into scripts; same script runs with different data sets
Keyword-driven testing	A test automation approach where test actions are expressed as readable keywords, separating test logic from implementation
Static analysis tool	A tool that analyzes source code without executing it, detecting defects and standard violations
Test management tool	A tool that supports planning, monitoring, controlling, and reporting on test activities
Performance testing tool	A tool that simulates load and measures system response times and behavior under stress
Model-based testing	An approach where test cases are automatically generated from a behavioral model of the system
CI/CD	Continuous Integration / Continuous Delivery — a DevOps practice where code changes trigger automated builds and tests
Coverage measurement tool	A tool that measures how much of the code or requirements is exercised by a test suite

Practice Questions

- What are the main categories of test tools? Give one example of what each supports.
- What is the difference between data-driven testing and keyword-driven testing?
- Name three benefits and three risks of test automation.
- Why does test automation require ongoing maintenance effort?
- What factors should be considered when selecting a test tool?
- What is a proof of concept in the context of tool selection, and why is it recommended?
- Why is a passing automated test suite not a guarantee of software quality?
- What is the difference between a static analysis tool and a test execution tool?
- How do CI/CD tools connect to the concept of shift left from Chapter 2?
- Which type of tool would you use to measure branch coverage after running your test suite?

ISTQB CTFL — Glossary

All key terms from Chapters 1–6, sorted alphabetically. Each entry shows the chapter(s) where it is tested and the cognitive level.

How to use: Read the term, try to recall the definition, then check it. Focus extra time on K2 terms (you need to *explain* them, not just recall) and K3 terms (you need to *apply* them).

A

Acceptance criteria Ch4 💡 K2 The conditions a user story must meet to be accepted by stakeholders. They define the scope of the story, guide acceptance testing, and are typically written in scenario-oriented (Given/When/Then) or rule-oriented format.

Acceptance testing Ch2 💡 K2 The test level focused on validation — confirming the system is ready for deployment. Performed by business representatives or end users. Includes UAT, OAT, contractual/regulatory testing, alpha, and beta testing.

Alpha testing Ch2 💡 K2 A type of acceptance testing performed by internal users at the developer's site, before release.

Anomaly Ch3 🧠 K1 Any result or condition that deviates from expectations, discovered during a review. May or may not be a defect — needs to be investigated.

ATDD — Acceptance Test-Driven Development Ch2 Ch4 💡 K2 An approach where test cases are written before the user story is implemented, based on acceptance criteria. Steps: specification workshop → test case creation → implementation. Closely related to BDD.

B

BDD — Behavior-Driven Development Ch2 🧠 K1 A test-first approach where tests are written in natural language using the Given/When/Then format, making them understandable to the whole team including non-technical stakeholders.

Beta testing Ch2 💡 K2 A type of acceptance testing performed by external users in their own environment, after release.

Black-box technique Ch4 💡 K2 A test technique that derives test cases from the specified behavior of the software — without knowledge of its internal structure. Tests survive code changes because they are specification-based. See also: *Equivalence Partitioning*, *Boundary Value Analysis*, *Decision Table Testing*, *State Transition Testing*.

Black-box testing Ch2 💡 K2 A test type where tests are based on the specification or specified behavior of the system, without knowledge of internal structure. Can be applied at any test level.

Boundary Value Analysis (BVA) Ch4 ⚙️ K3 A black-box technique that extends equivalence partitioning by specifically targeting the boundaries of ordered partitions. Two versions: 2-value BVA (boundary + nearest

neighbor) and 3-value BVA (boundary + value immediately below + value immediately above). Exploits the fact that developers make the most mistakes at the edges of ranges.

Branch Ch4  K2 A transfer of control between nodes in a control flow graph. Branches can be unconditional or conditional (e.g., from an if/else or switch statement).

Branch coverage Ch4  K2 The percentage of branches exercised by a test suite. 100% branch coverage guarantees 100% statement coverage (subsumption), but not vice versa. Stronger than statement coverage.

Branch testing Ch4  K2 A white-box technique that designs test cases to exercise all branches in the code. Corresponds to branch coverage.

C

Checklist-based testing Ch4  K2 An experience-based technique where tests are guided by a prepared list of conditions, often phrased as questions. Checklists should be regularly updated based on defect analysis to prevent them from wearing out.

Component integration testing Ch2  K2 A test level focused on testing the interfaces and interactions between integrated components. Usually performed by developers or testers after component testing.

Component testing Ch2  K2 The test level focused on individual components tested in isolation. Also called unit testing. Typically performed by developers.

Confirmation testing Ch2  K2 Testing performed after a defect has been fixed, to verify that the specific defect no longer causes a failure. Also called re-testing.

Configuration management (CM) Ch5  K2 The discipline of identifying, tracking, and controlling versions of testware and test objects. Ensures test cases can be linked to the correct software version and that testing is reproducible. Supporting context for FL-5.1.1 — not a standalone LO.

CI/CD — Continuous Integration / Continuous Delivery Ch6  K1 A DevOps practice where every code change automatically triggers a build and a test run. Integrates testing directly into the development pipeline, enabling fast feedback and supporting shift left. CI/CD tools connect developer commits to test results without manual intervention.

Coverage measurement tool Ch6  K1 A tool that measures how much of the code or requirements has been exercised by a test suite. Produces objective metrics such as statement coverage or branch coverage percentages.

Coverage criterion Ch4  K2 A rule or target used to determine whether a test suite is sufficient. Examples: all partitions exercised (EP), all branches exercised (branch coverage), all valid transitions exercised (state transition).

D

Data-driven testing Ch6  K1 A test automation approach where the test script is separated from the test data. Data is stored externally (e.g., a spreadsheet or database) and fed into the script at runtime, allowing the same script to run with many different data sets.

Decision table testing Ch4  K3 A black-box technique used to test combinations of conditions and their resulting actions. Each column in the table represents one rule (unique combination of conditions). Best suited

for testing business logic with complex conditional behavior.

Defect Ch1  K1 A flaw or imperfection in a work product (code, document, design) introduced by a human error. When executed, a defect may cause a failure — or may not, depending on the conditions.

Defect management Ch5  K3 The process of tracking defects from discovery to closure. Involves logging, classifying, prioritizing, fixing, retesting, and closing defects. The exam tests the content of a defect report (FL-5.4.1/5.4.2).

Defect report Ch5  K3 A document that records a defect, containing all information needed to reproduce, track, and resolve it. Key fields: unique ID, summary, date/author, environment, steps to reproduce, expected result, actual result, severity, priority, status, attachments.

Definition of Done (DoD) Ch5  K2 Agile equivalent of exit criteria — the conditions that must be met for a user story or iteration to be considered complete.

Definition of Ready (DoR) Ch5  K2 Agile equivalent of entry criteria — the conditions that must be met before work on a user story or iteration can begin.

Dynamic testing Ch1 Ch3  K2 Testing performed by executing the software. Defects are revealed through failures — the system must run. Contrast with static testing.

E

Entry criteria Ch5  K2 Conditions that must be satisfied before a test activity begins. Prevents wasted effort on testing when the system is not ready. In Agile: Definition of Ready (DoR).

Equivalence partitioning (EP) Ch4  K3 A black-box technique that divides input (or output) data into partitions where all values are expected to be treated identically by the system. One test case per partition is sufficient. Coverage = partitions exercised ÷ total partitions.

Error Ch1  K1 A human mistake — for example, a developer misunderstanding a requirement. An error introduces a defect into a work product.

Error guessing Ch4  K2 An experience-based technique that uses the tester's knowledge of how software typically fails to anticipate and target likely defects. Can be structured as a *fault attack* using a list of likely errors.

Exit criteria Ch5  K2 Conditions that determine when a test activity is sufficiently complete. May include coverage targets, defect density thresholds, or time/budget exhaustion. In Agile: Definition of Done (DoD).

Experience-based technique Ch4  K2 A category of test techniques based on the tester's knowledge, intuition, and experience. Complements black-box and white-box techniques. Includes error guessing, exploratory testing, and checklist-based testing.

Exploratory testing Ch4  K2 An experience-based technique where the tester simultaneously designs, executes, and evaluates tests while learning about the system. Uses a *test charter* to guide each time-boxed session. Most effective when specs are incomplete or time is limited.

F

Failure Ch1  K1 The visible, observable incorrect behavior of a system caused by the execution of a defect. Not all defects cause failures — some require specific conditions to trigger.

Fault attack Ch4 💡 K2

A structured form of error guessing: the tester creates a list of likely errors and defects, then designs tests specifically to expose them.

Functional testing Ch2 💡 K2 A test type that evaluates *what* the system does — its functional completeness, correctness, and appropriateness. Can be applied at any test level.

I

Impact analysis Ch2 💡 K2

The process of evaluating which parts of a system are affected by a proposed change. Used before maintenance testing to define the scope of regression testing. Relies on good traceability.

Informal review Ch3 🧠 K1 The least formal review type. No defined process, no required output. Goal is simply to find anomalies through a casual inspection.

Inspection Ch3 🧠 K1 The most formal review type. Follows a complete defined process based on ISO/IEC 20246. Maximum anomaly detection is the primary goal; metrics are collected. The author **cannot** act as review leader or scribe — unique to this review type.

INVEST Ch4 💡 K2 An acronym defining the characteristics of a good user story: **I**ndependent, **N**egotiable, **V**aluable, **E**stimable, **S**mall, **T**estable.

K

Keyword-driven testing Ch6 🧠 K1 A test automation approach where test actions are expressed as readable keywords (e.g., `Login`, `ClickButton`, `VerifyText`). The keywords are implemented separately, allowing non-programmers to write and maintain tests without scripting knowledge.

M

Maintenance testing Ch2 💡 K2 Testing performed on a system already in production when it needs to change. Triggered by modifications (enhancements, bug fixes, hot fixes), upgrades/migrations (new platform, OS, environment), or retirement (end-of-life data archiving).

Model-based testing Ch6 🧠 K1 A test approach where test cases are automatically derived from a behavioral model of the system. The model (e.g., a state machine or decision table) serves as the test basis; a tool generates test cases from it. Reduces manual test design effort for complex systems.

Moderator Ch3 🧠 K1 The role responsible for running a review meeting effectively. Handles facilitation, mediation, and time management, and ensures a safe environment for feedback. Required in technical reviews and inspections; optional in walkthroughs.

N

Non-functional testing Ch2  K2 A test type that evaluates *how well* the system performs — covering performance, security, usability, reliability, maintainability, portability, and safety. Can be applied at any test level.

O

OAT — Operational Acceptance Testing Ch2  K2 A type of acceptance testing focused on operational readiness: backup and recovery, security, performance under load, and other operational characteristics.

P

Priority Ch5  K2 How urgently a defect needs to be fixed, from a business perspective. High priority means it must be fixed soon — regardless of severity. A cosmetic bug on the login screen may be low severity but high priority.

Product risk Ch5  K2 The risk that a system component or feature will fail to meet expected behavior or quality. Product risks drive the scope and depth of testing.

Performance testing tool Ch6  K1 A tool that simulates concurrent user load on a system and measures response times, throughput, and resource consumption under stress. Used for load testing, stress testing, and endurance testing.

Project risk Ch5  K2 The risk to the project itself — schedule, budget, resources, or team. Managed through project management, not testing.

Q

Quality assurance (QA) Ch1  K1 A process-oriented, preventive activity that improves the development process to prevent defects from occurring. Distinct from testing, which is product-oriented and corrective. QA sets up the good process; testing checks the result.

R

Regression testing Ch2  K2 Re-running tests after a change to detect unintended side effects — ensuring that existing functionality has not been broken. Grows with every release and is a strong candidate for automation.

Review Ch3  K1 A form of static testing where one or more people manually evaluate a work product. Types range from informal to formal (inspection). Goal: find anomalies, improve quality, build shared understanding.

Review leader Ch3  K1 The role responsible for organizing the overall review: deciding who participates, when, and where. Cannot be the same person as the scribe or author in an inspection.

Risk   The possibility of an event occurring that causes an adverse consequence. Characterized by its likelihood and impact. The combination of both determines the risk level.

Risk assessment   The second stage of risk-based testing: evaluating identified risks by their likelihood and impact to determine which areas need the most testing attention.

Risk identification   The first stage of risk-based testing: brainstorming with stakeholders to identify what could go wrong in the system.

Risk-based testing   A testing approach that uses risk as the primary guide for deciding what to test, how much to test, and when to test. Ensures the highest-risk areas receive the most testing attention — especially important when time is limited.

Root cause   The fundamental underlying reason why an error was made — e.g., a misunderstood requirement, lack of training, or unclear documentation. Fixing root causes prevents future defects.

S

Scribe   The role responsible for recording anomalies, decisions, and new findings during a review meeting. Mandatory in walkthroughs and inspections.

Severity   The impact of a defect on the system — how badly it affects functionality, data, or operations. Classified as Critical, Major, Minor, or Trivial. Not the same as priority.

Shift left   The practice of moving testing activities earlier in the SDLC — performing test analysis, design, and even execution as early as possible to find defects when they are cheapest to fix.

SDLC — Software Development Lifecycle   The process used to plan, create, test, and deliver software. The choice of SDLC model (sequential, iterative, incremental) directly impacts how testing is structured and when it occurs.

State transition testing   A black-box technique that tests a system's behavior based on its states and the transitions between them. Coverage criteria: all states (weakest), all valid transitions / 0-switch (most common), all transitions including invalid (strongest).

Statement coverage   The percentage of executable statements exercised by a test suite. 100% statement coverage does not guarantee all branches are tested. Weaker than branch coverage.

Statement testing   A white-box technique that designs test cases to exercise executable statements. Corresponds to statement coverage.

Static analysis    Tool-assisted evaluation of a work product against a formal syntax — without executing it. Examples: linters, code analyzers. Finds issues like undefined variables, unreachable code, and coding standard violations. In Ch6, static analysis tools are a named tool category that supports static testing activities.

Static testing    Testing performed without executing the software. Includes both manual reviews and automated static analysis. Can find defects in non-executable work products (requirements, designs, test plans) that dynamic testing cannot reach.

System integration testing   A test level focused on the interfaces between the system under test and external systems or services. Tests interactions at the system boundary.

System testing   A test level that tests the end-to-end behavior of the entire system against its specified requirements. Typically performed by independent testers.

T

Test automation   **K1** The use of tools to execute tests and compare actual results to expected results automatically, without manual intervention. Most effective for repetitive, stable, high-volume testing. Not a replacement for human judgment and exploratory testing.

Test automation framework   **K1** A set of rules, libraries, and tools that provide the structure and conventions for writing, organizing, and running automated test scripts. Examples include data-driven and keyword-driven frameworks.

Test management tool   **K1** A tool that supports the planning, monitoring, controlling, and reporting of test activities. Typically manages test cases, test suites, test execution schedules, and defect tracking in one place.

TDD — Test-Driven Development   **K1** A development practice where developers write a failing unit test first, then write code to make it pass, then refactor. Implements the early testing principle.

Technical review   **K2** A semi-formal review type led by a trained moderator or peer leader with technically qualified reviewers. Focus is on achieving consensus on technical issues and detecting anomalies.

Test basis   **K1** All the information used to derive test cases — requirements, user stories, designs, code, risk assessments, etc. Good traceability links test cases back to the test basis.

Test charter   **K2** A document used in exploratory testing to define the objectives of a time-boxed test session. Guides exploration without over-constraining it.

Test completion report   **K2** A report produced at the end of a test phase or project. Summarizes what was tested, results, deviations from the plan, defect status, testware to archive, and lessons learned.

Test control   **K2** The activity of taking corrective actions when test monitoring reveals deviations from the test plan — such as reprioritizing tests, adjusting scope, or adding resources.

Test level   **K2** A group of test activities organized and managed together, with a specific focus and test basis. The five levels: component, component integration, system, system integration, and acceptance testing.

Test monitoring   **K2** The ongoing activity of checking actual test progress against the plan, using metrics. Enables test control when deviations are found.

Test object   **K1** The specific work product being tested — e.g., a component, a system, a document, or a configuration.

Test plan   **K2** A document that describes the objectives, scope, approach, schedule, and resources for a test effort. A living document updated as the project evolves.

Test progress report   **K2** A report that communicates the current testing status to stakeholders. Covers the period, progress vs. plan, impediments, and planned next steps.

Test pyramid   **K2** A model recommending a distribution of tests across levels: many unit tests (fast, cheap), fewer integration tests, and very few UI/end-to-end tests (slow, brittle). Encourages automating at the lowest effective level.

Test type   **K2** A classification of testing by what aspect of the system is being tested: functional, non-functional, black-box, or white-box. All four types can be applied at any test level.

Testware   **K2** All artifacts produced by test activities — including test plans, test cases, test scripts, test data, test logs, defect reports, and completion reports.

Testing quadrants  Ch5  K2 A framework (Brian Marick) for categorizing test types by two axes: technology-facing vs. business-facing, and supporting the team vs. critiquing the product. Q1 (tech, support): unit/integration tests. Q2 (business, support): functional/story tests. Q3 (business, critique): exploratory/UAT. Q4 (tech, critique): performance/security.

U

UAT — User Acceptance Testing  Ch2  K2 A type of acceptance testing where real users validate that the system meets their needs and is ready for use in production.

User story  Ch4  K2 A description of a feature that delivers value to a user or purchaser. Written as: "As a [role], I want [goal], so that [business value]." Has three components: Card, Conversation, Confirmation (the 3 Cs). Should follow the INVEST criteria.

V

Validation  Ch1  K1 Confirming that a product meets the needs of the user and other stakeholders. Asking: "Are we building the right product?" Focuses on fitness for purpose.

Verification  Ch1  K1 Confirming that a work product meets its specified requirements. Asking: "Are we building the product right?" Focuses on conformance to specifications.

W

Walkthrough  Ch3  K1 A review type led by the author, who guides participants through the work product. Aims to educate, gain consensus, and detect anomalies. Individual review beforehand is optional; scribe is mandatory.

White-box technique  Ch4  K2 A test technique that derives test cases from the internal structure of the software — code paths, branches, data flows. Tests depend on the implementation. Includes statement testing and branch testing.

White-box testing  Ch2  K2 A test type where tests are based on the internal structure or implementation of the system. Can be applied at any test level.

ISTQB CTFL — Common Mistakes

Typical exam traps, easily confused concepts, and tips to avoid falling for them.

Easily Confused Concepts

Error vs. Defect vs. Failure vs. Root Cause Ch1

Term	What it is	Who creates it
Error	A human mistake	Developer, analyst, tester
Defect	The flaw the error introduced into the work product	Caused by the error
Failure	The visible incorrect behavior when the defect is executed	Caused by the defect
Root cause	The underlying reason the error was made	Process or environment issue

Trap: "A defect always causes a failure." — FALSE. A defect may never execute, or may only fail under specific conditions.

Testing vs. Quality Assurance Ch1

	Testing	QA
Focus	Product	Process
Goal	Find defects	Prevent defects
Orientation	Corrective	Preventive

Trap: "QA and testing are the same thing." — FALSE. Testing is part of QA, but QA is broader — it covers the entire development process.

Verification vs. Validation Ch1

- **Verification** → "Are we building the product **right**?" — conformance to specification
- **Validation** → "Are we building the **right** product?" — fitness for user needs

Memory tip: Validation = Value to the user.

Confirmation Testing vs. Regression Testing ch2

- **Confirmation testing:** re-tests a specific defect that was fixed — did *this* fix work?
- **Regression testing:** re-tests the whole system after any change — did anything *else* break?

Trap: "Regression testing is only needed after bug fixes." — FALSE. Any change (enhancement, refactor, configuration change) can introduce regressions.

Black-box / White-box as Test Types vs. Test Techniques Ch2 Ch4

The same terms appear in two different chapters with related but distinct meanings:

Context	Meaning
Ch2 — Test types	<i>What</i> the tests are based on: specification (black-box) vs. structure (white-box)
Ch4 — Test techniques	<i>How</i> test cases are derived: from spec (black-box techniques) vs. from code (white-box techniques)

Trap: Seeing "black-box" in a question and assuming it always means the Ch4 technique. Check what the question is actually asking about.

Static Testing vs. Static Analysis Ch3

- **Static testing** = the broad category — evaluating a work product *without executing it* (includes reviews AND tool-based analysis)
- **Static analysis** = the tool-based subset of static testing — automated analysis against a formal syntax

Trap: "Static analysis and static testing are the same." — FALSE. Static analysis is *one type* of static testing.

Walkthrough vs. Technical Review vs. Inspection Ch3

	Walkthrough	Technical Review	Inspection
Led by	Author	Moderator or peer leader	Moderator (NOT author)
Formality	Low–Medium	Medium–High	Highest
Scribe	Mandatory	Depends	Mandatory
Author as leader?	Yes	No	Never

Trap: "In a walkthrough, individual review beforehand is mandatory." — FALSE. It is *optional*. **Trap:** "In an inspection, the author can lead the review." — FALSE. The author cannot be the review leader or the scribe.

Equivalence Partitioning vs. Boundary Value Analysis Ch4

- **EP** — selects *one representative value* from each partition (any value, not necessarily the edge)
- **BVA** — specifically targets the *edges* of ordered partitions; extends EP, doesn't replace it

Trap: "BVA replaces EP." — FALSE. BVA extends EP by adding boundary-specific test cases on top of it.

2-value BVA vs. 3-value BVA Ch4

- **2-value:** boundary value + its closest neighbor from the adjacent partition
- **3-value:** the boundary value itself + the value immediately below it + the value immediately above it

Trap: "3-value BVA tests three partitions." — FALSE. It tests three specific values around one boundary.

Statement Coverage vs. Branch Coverage Ch4

- **Statement coverage:** every executable statement run at least once
- **Branch coverage:** every branch (true/false outcome of every decision) exercised

Critical rule: Branch coverage **subsumes** statement coverage — 100% branch coverage guarantees 100% statement coverage. The reverse is NOT true. **Trap:** "100% statement coverage means all branches are tested." — FALSE.

Severity vs. Priority Ch5

- **Severity** = how badly the defect impacts the system (technical view)
- **Priority** = how urgently it needs to be fixed (business view)

Trap: "A high-severity defect is always high priority." — FALSE. Example: a crash in a feature used by 0.1% of users = high severity, low priority. Example: a typo on the login page = low severity, high priority.

Test Monitoring vs. Test Control Ch5

- **Monitoring** = collecting data and checking progress against the plan (observing)
- **Control** = taking corrective action when deviations are found (acting)

Trap: "Monitoring and control are the same activity." — FALSE. Monitoring without control is just observation.

Product Risk vs. Project Risk Ch5

- **Product risk** = risk that the system fails to meet quality expectations → drives *testing* decisions
- **Project risk** = risk to the project schedule, budget, or resources → managed by *project management*

Exam Traps — Statements That Look True But Aren't

Trap statement	Why it's wrong
"Testing proves the software is correct."	Testing shows the <i>presence</i> of defects, never their absence (Principle 1)
"With enough time, we can test everything."	Exhaustive testing is impossible (Principle 2)
"If all tests pass, the software has no defects."	Absence of defects is a fallacy (Principle 7)
"More testing always means higher quality."	Testing is context-dependent; more is not always better (Principle 6)
"The author can lead or scribe an inspection."	Not allowed — unique rule for inspections
"Q1 in the testing quadrants is business-facing."	Q1 is <i>technology-facing</i> + supporting the team; Q2 is business-facing
"Regression testing only runs after a bug fix."	Any change — enhancements, refactors, migrations — can break existing behavior
"100% statement coverage guarantees full testing."	It does not test all branches; branch coverage is stronger
"Exploratory testing has no structure."	It uses time-boxed <i>sessions</i> guided by a <i>test charter</i>
"0-switch coverage tests sequences of transitions."	0-switch = single transitions only, no chaining
"White-box testing is always dynamic."	White-box <i>analysis</i> can be applied statically to pseudocode or unexecutable code
"Wideband Delphi is a named CTFL 4.0 technique."	Only Planning Poker is named; Wideband Delphi is from older versions
"A walkthrough requires individual review beforehand."	Individual review is <i>optional</i> in walkthroughs; only the scribe is mandatory
"BVA can be applied to any partition."	BVA only applies to <i>ordered</i> partitions (those with min/max values)

Tips to Avoid Traps

Read the question stem carefully. ISTQB questions often hinge on a single word: *BEST*, *MOST appropriate*, *EXCEPT*, *NOT*, *LEAST likely*. Missing that word flips the correct answer.

Know the difference between K-levels in the question. - A K1 question asks you to *identify* or *recall* — pick the correct definition. - A K2 question asks you to *explain* or *distinguish* — pick the most accurate explanation. - A K3 question gives you a *scenario* — apply the technique and derive the result.

When two answers both seem correct, go more specific. ISTQB prefers the answer that matches the syllabus definition most precisely. The "almost right" answer is usually a distractor that swaps one key word.

For K3 technique questions, work it on paper. Don't try to do EP, BVA, or state transition in your head. Sketch the partitions or table quickly — you'll catch errors you'd miss mentally.

The syllabus definition wins over industry practice. If you've worked in QA for years and something "feels" right, but it contradicts the CTFL 4.0 syllabus definition, go with the syllabus. The exam tests the standard, not real-world variation.

Watch for absolutes. Words like *always*, *never*, *all*, *only*, *impossible* are often traps. Testing principles 1 and 2 specifically address what is and isn't possible in testing.

When unsure between two answers, eliminate first. Cross out the two clearly wrong answers. Then compare the remaining two against the exact syllabus wording — the right answer will use the correct terminology.